



Singleton

Padrões de Projeto - Aula #06

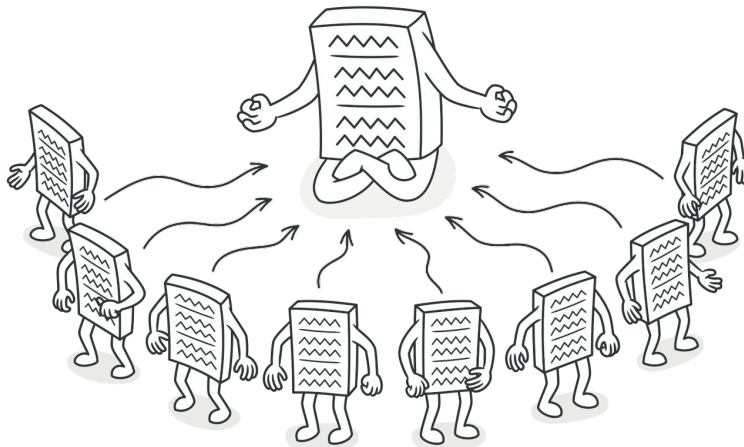
Prof. Dr. Carlos Melo

 carlos.melo@upe.br

Padrões Criacionais (#2)

Singleton

Garanta que uma classe tenha uma única instância e um ponto global de acesso a ela



- O Singleton é o padrão mais criticado do catálogo

Objetivos de aprendizagem

- Implementar o Singleton de três formas e saber qual usar.
- Entender o porque do Singleton ser um considerado um antipadrão.
- Módulos em Python já são Singletons, o que quase sempre tornam o uso do padrão desnecessário.

Singleton - O Problema

- O acervo precisa falar com o banco de dados.
- Cada Conexao abre um *socket*, autentica e negocia o protocolo
- 3*1,5s (handshake)
- O número máximo de conexões no Postgres é 100.
- Dirty Read o tempo todo.
- Conexão é recurso compartilhado, não deveria ser possível ter dois objetos representando o mesmo recurso

```
1 class ConexaoBD:
2     def __init__(self, dsn: str):
3         print(f"[REDE] abrindo conexão com {dsn}...")
4         time.sleep(1.5)
5         self._socket = f"socket-{id(self)}"
6
7     def consultar(self, sql: str):
8         return f"[{self._socket}] {sql}"
```

```
1 # exportador.py
2 bd = ConexaoBD("postgres://acervo")
3 # processador.py
4 bd = ConexaoBD("postgres://acervo")
5 # notificador.py
6 bd = ConexaoBD("postgres://acervo")
```

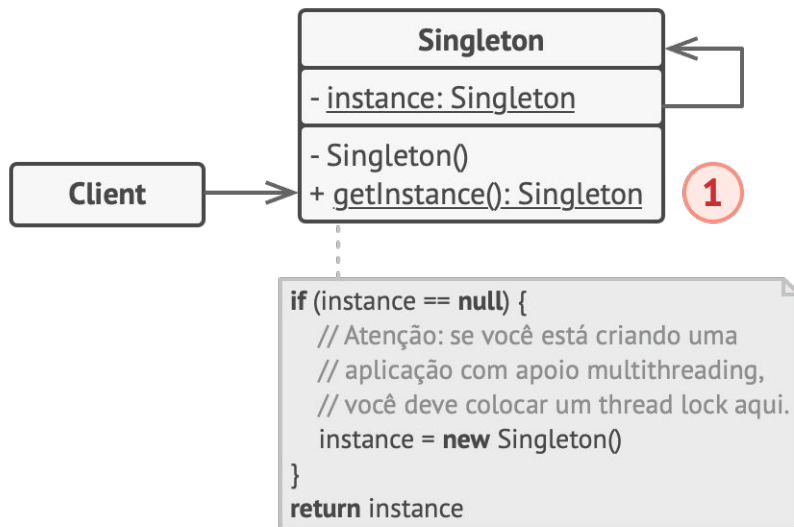
Singleton - Intenção

"Garantir que uma classe tenha somente uma instância e fornecer um ponto global de acesso a ela."

- Padrão criacional
 - O menor e mais controverso dos padrões
- Promessas:
 - Uma única instância (o inferno na Terra de testar)
 - Ponto global de acesso (é literalmente uma variável global)
- Torna o construtor inacessível, e um método estático assume o lugar dele (cria na primeira chamada) e depois só retorna a instância caso não exista.

Singleton - Estrutura

1. Singleton declara o método estático `getInstance` que retorna a mesma instância de sua própria classe.



Implementando o Singleton #1

- Sobrescrevendo o `__new__`

```
1 class ConexaoBD:
2     _instancia = None
3
4     def __new__(cls, *args, **kwargs):
5         if cls._instancia is None:
6             cls._instancia = super().__new__(cls)
7             return cls._instancia
8
9     def __init__(self, dsn="postgres://acervo"):
10        if hasattr(self, "_socket"):
11            return
12        print(f"[REDE] abrindo conexão com {dsn}...")
13        self._socket = f"socket-{id(self)}"
```

```
1 a = ConexaoBD() # [REDE] abrindo conexão...
2 b = ConexaoBD() # cri cri cri
3 print(a is b) # True
```

Implementando o Singleton #2

- Metaclasses com Lock

```
1 import threading
2
3 class SingletonMeta(type):
4     _instancias = {}
5     _lock = threading.Lock()
6
7     def __call__(cls, *args, **kwargs):
8         with cls._lock:
9             if cls not in cls._instancias:
10                 cls._instancias[cls] =
11                 super().__call__(*args, **kwargs)
12                 return cls._instancias[cls]
13
14 class ConexaoBD(metaclass=SingletonMeta):
15     def __init__(self, dsn="postgres://acervo"):
16         print(f"[REDE] abrindo conexão com {dsn}...")
17         self._socket = f"socket-{id(self)}"
```

- O Lock é obrigatório em código com threads.
- Sem ele, duas threads podem passar pelo **if** simultaneamente e abrir duas conexões, literalmente o que o Singleton deveria impedir.

Implementando o Singleton #3

- O módulo já é um Singleton

```
1 # conexao.py
2 print("[REDE] abrindo conexão com postgres://acervo...")
3 _socket = _abrir_socket("postgres://acervo")
4
5 def consultar(sql: str):
6     return f"[{_socket}] {sql}"
7
8 # exportador.py
9 from conexao import consultar # [REDE] abrindo conexão...
10
11 # processador.py
12 from conexao import consultar # cri cri cri
13
14 import conexao, sys
15 print(sys.modules["conexao"] is conexao) # True
```

- O módulo é Thread-safe, então não precisa de Lock explícito.
- Continua sendo horrível de testar.

Por quê o Singleton é chamado de Antipadrão?

- É uma variável global como roupa nova.
- Ponto único de falha de acesso global.
- Difícil de se testar de forma unitária. Geralmente o teste vira teste de integração.
- Viola o S do SOLID, uma vez que a classe gerencia a conexão e controla seu próprio ciclo de vida.
- Sem o Lock o próprio Singleton falha em garantir unicidade.
- Esconde dependências:



```
1 class Exportador:
2     def exportar(self, imagens):
3         dados = ConexaoBD().consultar("SELECT * FROM acervo") # invisível
```



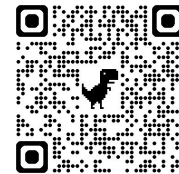
```
1 def test_exportar():
2     Exportador().exportar(imagens) # Abre conexão real com o banco
```

Então... quando usar?

```
1 class Exportador:
2     def __init__(self, bd: ConexaoBD): # dependência VISÍVEL
3         self._bd = bd
4
5     def exportar(self, imagens):
6         dados = self._bd.consultar("SELECT * FROM acervo")
```

```
1 bd = ConexaoBD("postgres://acervo") # Só uma vez
2 Exportador(bd).exportar(imagens)
3 Processador(bd).processar(imagens)
```

- Logger (quando o acoplamento for barato).
- *Pool* de conexões gerenciado pelo framework
 - O *pool* é singleton, as conexões não.
- Cache de processo sem requisito de isolamento entre testes.



Prof. Dr. Carlos Melo
<https://calendly.com/prof-casm>

 carlos.melo@upe.br